

Report Task 2

Artificial Intelligence and Expert Systems

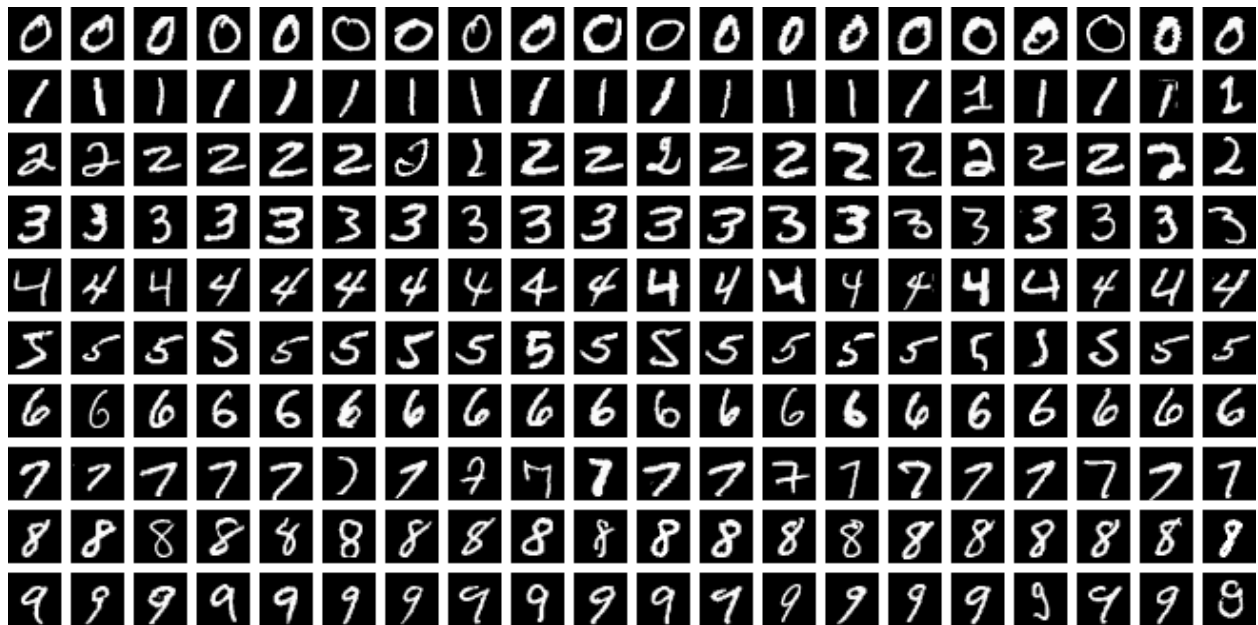
Stanisław Marciniak 247032

Krzysztof Muszyński 247035

1. Dataset

MNIST (Modified National Institute of Standards and Technology database) dataset was used for this project. It consists of 70,000 grayscale images of handwritten digits (0-9), each the size of 28x28 pixels. The dataset was split into 60,000 training images and 10,000 test images.

Dataset link: <https://www.kaggle.com/datasets/hojjatk/mnist-dataset>



2. Model

Architecture

A neural network was implemented using Keras Sequential API. The architecture consists of:

- Input layer:

A *Flatten* layer converts each input image of size 28x28 pixels into a one-dimensional vector of 784 elements.

- Hidden layers:

The model includes five Dense layers with 128 neurons each, using the ReLU (Rectified Linear Unit) activation function. The choice of multiple layers increases the model's capacity to capture hierarchical patterns in the handwritten digits.

- Dropout layer:

A dropout layer was added to prevent overfitting by randomly deactivating 50% of neurons during training. This encourages the model to generalize better and reduces its dependency on specific neurons.

- Output layer:

The final *Dense* layer contains 10 neurons, each corresponding to one possible digit (0-9). A softmax activation function converts the outputs into probabilities (all 10 values sums to 1).

Optimization and Loss Functions

The model optimization was performed using the **Adam (Adaptive Moment Estimation)** optimizer. Adam combines the advantages of two popular methods: *Momentum* and *RMSProp*, by maintaining both an exponentially decaying average of past gradients and their squared values.

The loss function used was **Sparse Categorical Cross-Entropy**, which measures the dissimilarity between the predicted probability distribution $y^{\wedge}$ and the true class label y (encoded as an integer).

This formulation is suitable when labels are integer-encoded, unlike standard categorical cross-entropy which expects one-hot encoding.

Training Procedure

The training process consisted of 50 epochs and a batch size of 32.

3. Data preprocessing

Our image preprocessing included transforming the user drawn images to the same format in which images were fed to the model during training. This greatly improved our model's prediction accuracy.

The process consisted of the following stages:

Image Decoding

The server first transforms the Base64-encoded format image into a numerical representation

Grayscale Conversion

The image was converted to a single-channel grayscale format. Since the MNIST dataset consists of only grayscale images.

Digit Localization and Cropping

The algorithm detected the region containing the drawn digit and cropped it from the background.

Resizing and Centering

The cropped digit was resized to a 20×20 pixel size and placed in the middle of a 28x28 canvas. This ensured the hand drawn digit was placed similarly to those given to the model during training.

4. Model's accuracy

The model demonstrated excellent performance during both training and evaluation.

Over 50 epochs, training and validation accuracies increased steadily.

This indicated that the model learned effectively and there were no signs of overfitting.

Finally, the model achieved the following results:

- **Training Accuracy:** 99.73%
- **Validation Accuracy:** 97.93%
- **Test Accuracy:** 97.97%
- **Test Loss:** 0.25

These outcomes confirm that the neural network generalizes well, and predicts effectively.

To assess the models performance we can use the below metrics: precision, recall and F1-score. In simple terms:

- precision: measures how rarely the model missclassifies other digits as this digit.
- recall: measures how many of that digits were correctly reckognized
- F1-score: is a combination of precision and recall, its a balanced metric that considers both false positives and false negatives

Our model achieved an average precision, recall and F1-score of around 0.98 on the given dataset. This demonstrates that it very accurately predicts digits.

A slightly lower score: 0.97 appeared for digit '8', it might be due to the visual similarity with other digits such as '3' or '5'.

Overall the results indicate that the model performs very well with predicting digits from the MNIST dataset.

```

=== Evaluating on Test Data ===
Test Loss: 0.2518
Test Accuracy: 0.9797

=== Final Training Metrics ===
Training Loss: 0.0122
Training Accuracy: 0.9973
Validation Loss: 0.2607
Validation Accuracy: 0.9793

=== Classification Report ===

```

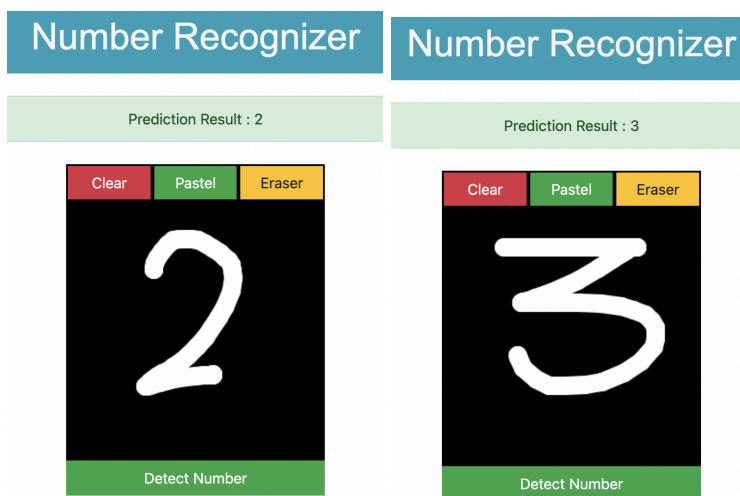
	precision	recall	f1-score	support
0	0.97	0.99	0.98	980
1	0.99	0.99	0.99	1135
2	0.97	0.98	0.98	1032
3	0.97	0.99	0.98	1010
4	0.99	0.97	0.98	982
5	0.99	0.97	0.98	892
6	0.99	0.98	0.98	958
7	0.98	0.98	0.98	1028
8	0.97	0.97	0.97	974
9	0.98	0.98	0.98	1009
accuracy			0.98	10000
macro avg	0.98	0.98	0.98	10000
weighted avg	0.98	0.98	0.98	10000

5. Conclusions

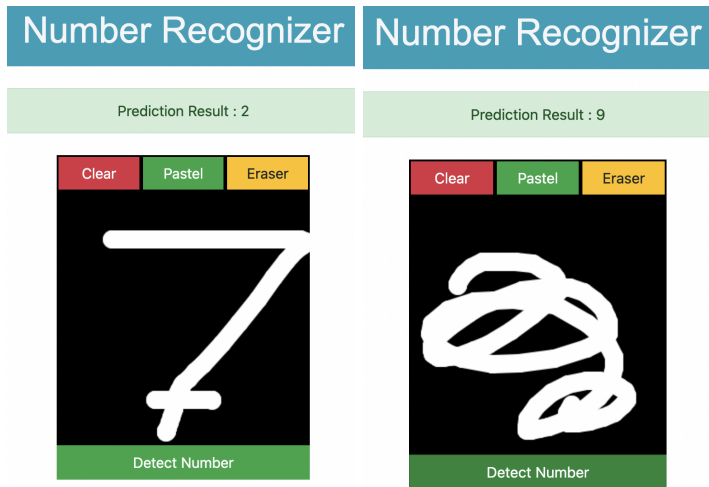
The application functions as intended -users can draw numbers directly on a canvas rendered in the web browser, and the connected model successfully predicts which number has been drawn.

The model's accuracy is strong; predictions are almost always correct when the digits are drawn clearly and neatly.

Examples of correct predictions:



Examples of incorrect predictions:



When the drawing is sloppy or ambiguous, the model's predictions tend to deviate.

Overall, the model demonstrates solid performance and behaves as expected.